

Recommending Restaurants through Personalized Rating Score Prediction, using Google Local Data

Abstract—With the rise of online platforms for business reviews, leveraging this data for personalized recommendations has become crucial for enhancing user experience. This paper presents a recommendation system built upon the Google Local Review dataset, specifically on the restaurants of Washington D.C. The dataset used filters out other business establishments so that only restaurants are analyzed, which is over 347,232 reviews for 3,446 restaurants. Utilizing a split of 80% for training, 10% for testing, and 10% for validation, we employ various predictive models including a bias model, Latent Factor Model, and regression-based approaches, aiming to predict user-specific rating scores. Our findings demonstrate the success of integrating user and restaurant biases into predictive modeling. This provides an approach to personalized restaurant recommendations. This concludes with insights drawn from our models and their implications for future recommendation systems and data-driven strategies in the restaurant industry.

Index Terms—Personalized Recommendations, Rating Prediction, Google Local Reviews, Recommender Systems, Predictive Modeling, Latent Factor Model, Data Engineering, User Bias, Restaurant Bias, Machine Learning.

I. DATASET

In our study, we utilized the Google Local Review (2021) Washington D.C. dataset to construct a recommendation system for restaurants within the city. The dataset is made up of two files: one containing the review data for locally reviewed businesses in the D.C. area, and the other providing metadata for each restaurant in the reviewed list. By leveraging the "category" feature and keywords like "restaurant," "pizza," and "bar," we successfully isolated the dataset to include only restaurants and food-related businesses for our analysis. The original dataset had 564,782 initial reviews and 11,059 business. The dataset that we filtered has 347,232 reviews for 3446 restaurants. We split the data up into 80 percent training set, 10 percent test set, and 10 percent validation set.

This dataset included aspects such as ratings and other relevant factors, enabling us to develop a robust recommendation system for dining establishments in Washington D.C. Below is the Review Data Schema:

```
{
  "user_id": "108642560086289718425",
  "name": "Keshava Mysore",
  "time": 1510920735052,
  "rating": 4,
  "text": "High prices for not so big portions!
  But I think it is the only decent food available
  on the National Mall area without walking a few
  miles away from there.",
  "pics": null,
  "resp": null,
  "gmap_id": "0x89b7b7851b06ef6b:0x5f356b1eb1da27"
}
```

The Review Data Schema is a structured representation of a single review entry from the dataset. Below is the Metadata Review Schema (one restaurant):

```
{
  'name': 'Water Bar DC',
  'address': 'Water Bar DC, 2822 Georgia Ave NW,
  Washington, DC 20001',
  'gmap_id': '0x89b7c9087b09e427:0
  xaf25f277cf2dd3c1',
  'description': None,
  'latitude': 38.92675,
  'longitude': -77.0231953,
  'category': ['Bar'],
  'avg_rating': 2.6,
  'num_of_reviews': 8,
  'price': None,
  'hours': [['Wednesday', '12 10PM '],
  ['Thursday', '12 10PM '],
  ['Friday', '12 P M 2AM '],
  ['Saturday', '12 P M 2AM '],
  ['Sunday', '1 7PM '],
  ['Monday', '12 10PM '],
  ['Tuesday', '12 10PM ']],
  'MISC': {'Service options': ['Delivery', '
  Takeout', 'Dine-in'],
  'Highlights': ['Fast service'],
  'Offerings': ['Alcohol', 'Beer', 'Cocktails', '
  Hard liquor', 'Wine'],
  'Amenities': ['Restroom'],
  'Atmosphere': ['Casual'],
  'Crowd': ['Groups']},
  'state': 'Open Closes 10PM',
  'relative_results': ['0x89b7b83debd5a20d:0
  xfac9686d95ac4053',
  '0x89b7b7cf13a507dd:0x1f0e3b369080f238'],
  'url': 'https://www.google.com/maps/place//
  data=!4m2!3m1!1s0x89b7c9087b09e427:0
  xaf25f277cf2dd3c1?authuser=-1&hl=en&gl=us'
}
```

A. Count of Ratings (Fig 1.)

This visualization shows the count of ratings (1-5 stars). We can see that there are more positive ratings than negative ratings. This means that users are more likely to leave a review if they have a good experience. Positive experiences tend to motivate users to share favorable opinions and recommend the restaurant. Conversely, this means if a user has a negative experience, they are less likely to leave a review. This skew introduces potential bias in the overall rating. For restaurants, it is important that customers who feel all ways review their experience for a more accurate interpretation of their rating.

B. Distribution of Number of Reviews Per User (Fig 2.)

This visualization explains how many reviews users give. We can see that 49.1 percent of users have given less than 10

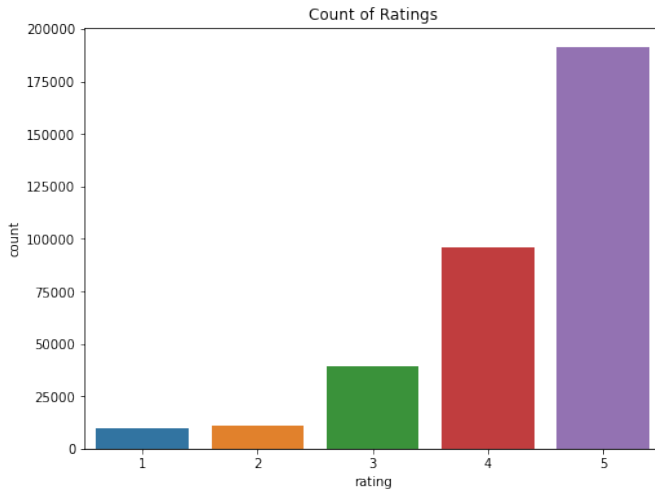


Fig. 1: Count of Ratings

reviews. This means that about half of users are low frequency reviewers. This group only leaves reviews occasionally and for specific needs. Conversely, 2.5 percent of users have given 50+ reviews. This group is significantly more engaged and most likely power users. They are likely to impact other users significantly and are the most influential on the platform. This visualization highlights a common pattern in many online platform reviewers. This structure of the dataset affected our feature engineering and explained low change in our performance metric when using Jaccard similarity.

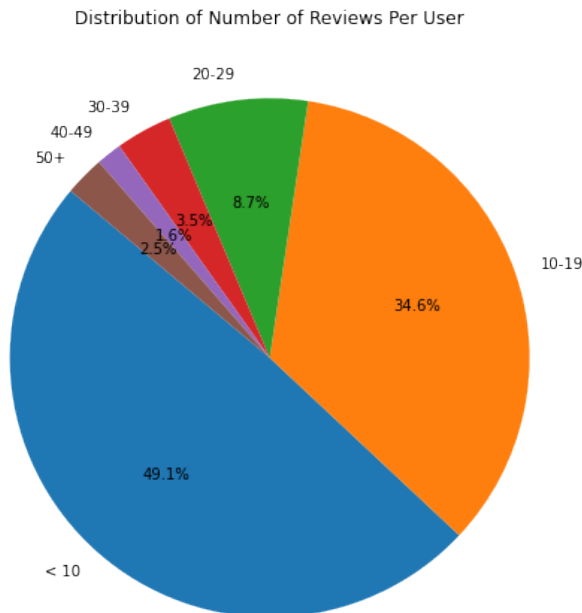


Fig. 2: Distribution of Number of Reviews Per User

C. Average Ratings Over Time and Missing Data (Fig. 3 and Fig. 4)

This graph shows an upward trend in ratings post 2009. This could suggest improvements in the quality of restaurants being reviewed, changes in review behavior, or shifts in user base. Data was sparse before 2009 which can be seen from fewer reviews made/collected. Spikes/drops in the graph can indicate real world events affecting reviews, changes in rating system, or data collection errors. Google reviews also started in 2007, this means that there is not data before this time. The first few years after 2007 must be slow because users are still adapting to this new feature.

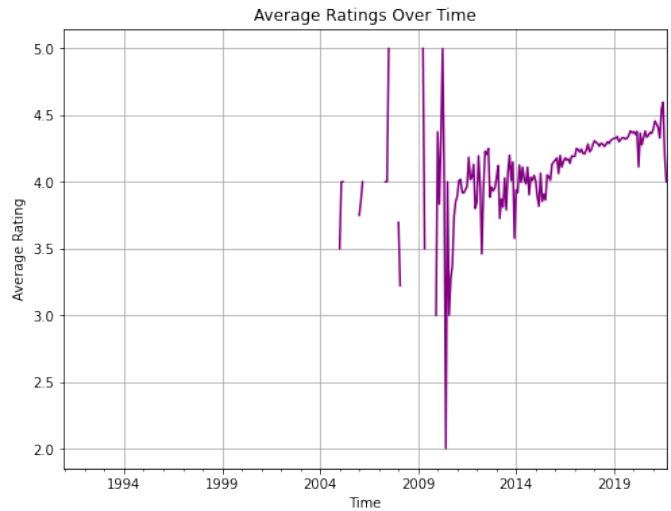


Fig. 3: Average Ratings Over Time

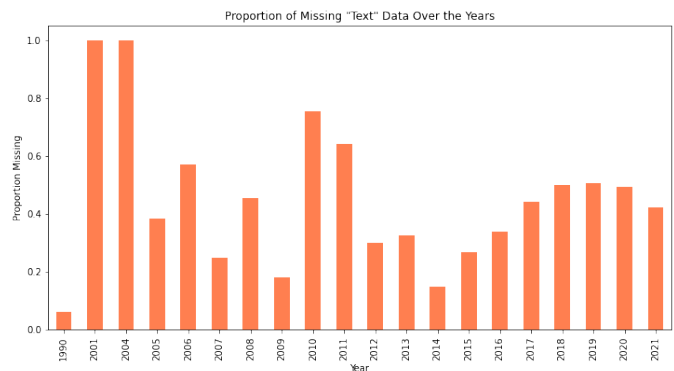


Fig. 4: Missing Data

D. Top 10 Most Common Words in Reviews (Fig. 5)

This bar chart shows the 10 most common words in the reviews text data. Common words like "food", "service", and "place" are used in reviews. Words "great" and "good" are very common because most of the reviews are positive (3+ star). Figure 4 helps us grasp the most prevalent themes in customer reviews (most of which are positive).

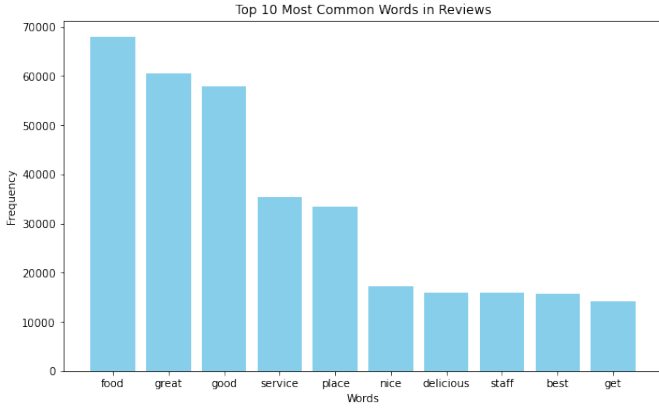


Fig. 5: Top 10 Most Common Words in Reviews

E. 1 Star Words vs. 5 Star Words (Fig. 6)

These two bar charts represent the top 10 most common words found in 1 star reviews and 5 star reviews. They provide a comparative analysis of language used by customers in reviewers under opposite extremes (5 stars and 1 star). The 1 star visualization suggests common themes of dissatisfaction related to aspects of their dining experience, while the 5 star visualization shows the opposite. The context of "good" showing up in the 1 star graph means "not good" if filtering two words. This requires using context clues.

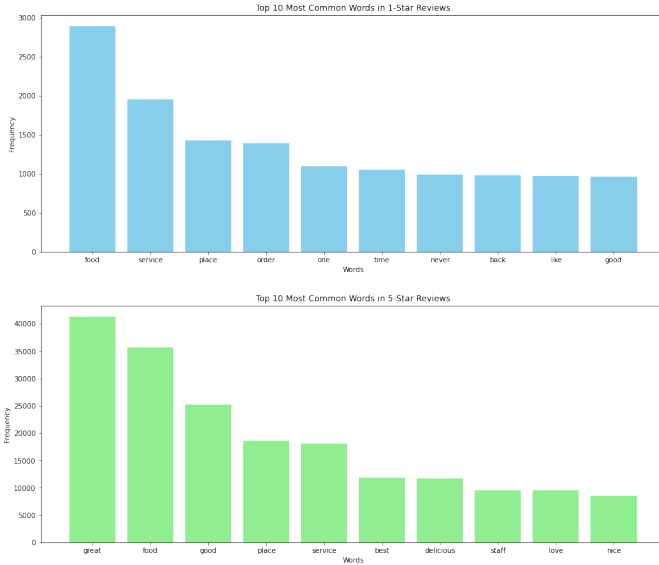


Fig. 6: 1 Star Words vs. 5 Star Words

F. Heatmap (Fig. 7)

The heat map below shows the count of reviews by year and rating, with years on the y-axis and ratings on the x-axis. The color intensity in each of the cells responds to number of reviews (darker colors indicate higher count in reviews. We can see some trends discussed in the previous visualizations in this heatmap. There is an increasing volume of reviews over

time, disproportionately larger amount of positive reviews, data sparsity in earlier years, and bias with positive reviews.

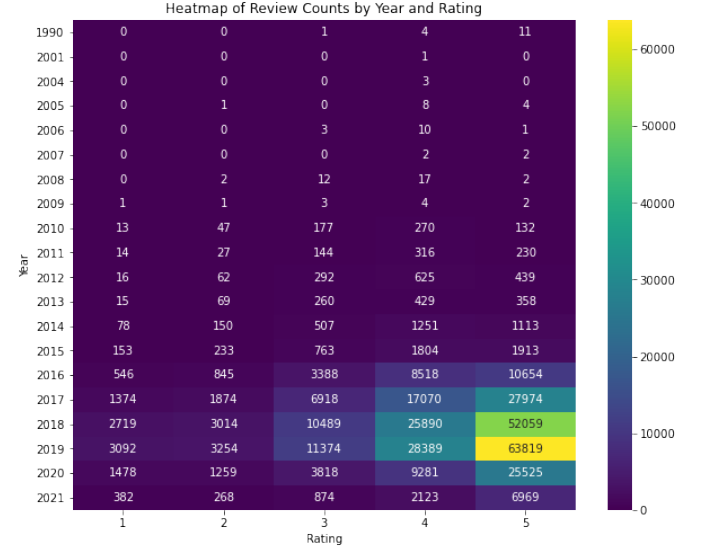


Fig. 7: Heatmap of counts of reviews by year and rating

II. PREDICTIVE TASK

The predictive task we analyze in this paper is the following: for a specific user and restaurant, predict a real valued rating ranging between 1 and 5 that the user is likely to give to that restaurant. In effect, we are modeling:

$$f(\text{user}, \text{restaurant}) = \text{rating} \in \mathbb{R} \quad (1)$$

A. Evaluation Metrics

We decided to approach our predictive task as a regression problem rather than a classification problem of the 5-star reviews. We chose this as we had more representative predictions which allowed for a sliding scale between star reviews. As a result, we used Mean Squared Error (MSE) for our performance metric. Its important to note that our data is not normally distributed, however, the MSE still gives us a good understanding of our errors in our prediction across the whole dataset and across different models.

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (2)$$

B. Baseline Models

We used a straightforward approach to creating our *Baseline* model and used the mean rating of every restaurant in the training set as the prediction for each user in our testing set. It's important to note that this is essentially equal to the variance of the model and is thus a very trivial predictor. Our MSE for the *Baseline* is 0.9641, setting our performance baseline to beat for our predictive task.

Mean Rating Baseline Model:

$$f(\text{user}, \text{restaurant}) = \frac{1}{N} \sum_{i=1}^n r_i \quad (3)$$

C. Assessing Validity

In order to assess our validity we are using the MSE as a measure of how good our model is. The lower the MSE, the better the model. In order to evaluate this in a consistent matter we randomized the order of the review data and then split our data into three sets, the training set (80%), the validation set (10%), and the testing set (10%).

D. Features & Model Selection

Given our baseline, we began exploring prediction models for interaction data between users and restaurants. We tested the *Bias* model and Latent Factor Model (*LFM*) for direct interaction data, meaning given a user, restaurant, and a rating. For real valued predictions, we tested linear regression, random forest regression, and xgboost (for exploratory purposes). For these models, we used feature engineering to gain information about interaction data as well as sentiment. We did this by finding user averages for rating restaurants as a feature and the restaurant rating average for all users that rated the restaurant.

III. MODEL RESULTS

A. Model 1: Bias Model

The bias model serves as a natural transition from using the naive baseline model of simply guessing the mean. Instead of having a single term hardcoded term for rating prediction we move to a set of variable terms as shown below:

$$r_{ui} = \alpha + \beta_u + \beta_i \quad (4)$$

r_{ui} is the predicted rating given the specific user and restaurant. α is the global bias term which we initially set as the mean rating across all the reviews from the training data. β_u is the user bias term. This term represents the users tendency to rate a restaurant higher or lower, the deviation from the average user rating. β_i is the restaurant bias term. This term represents the restaurant's deviation from the average restaurant rating. Below is a table of selected lambda values to show the effect on the MSE.

Lambda	MSE
1×10^{-6}	0.8089
1×10^{-5}	0.7688
1×10^{-4}	0.9443

Mean Squared Error (MSE) for Different Models

B. Model 2: Latent Factor Model

The Latent Factor Model is an expansion of the bias model. The "latent factors" refer to the underlying factors that determine how a user might rate a restaurant. The idea is that although these factors might not be directly observable, they can be inferred from a user-restaurant interaction.

$$r_{ui} = \alpha + \beta_u + \beta_i + \gamma_u \cdot \gamma_i \quad (5)$$

Most of the variables are the same as the bias model the only difference being γ_u and γ_i . γ_u represents the user latent factor vector while γ_i represents the restaurant latent factor vector. These vectors are initially defined randomly and get learned during the training process. In the model they get multiplied to signify the interaction between the user and the restaurant which should effect the rating prediction. Below is a table of selected lambda values and gamma size to show the effect on the MSE.

Lambda	Gamma Size	MSE
1×10^{-5}	1	0.7913
1×10^{-5}	3	0.8010
1×10^{-5}	5	0.8377
1×10^{-4}	1	0.9174
1×10^{-6}	1	0.9286

Mean Squared Error (MSE) for Different Models

C. Model 3: Linear Regression

The linear regression model is used to compare iterative gradient descent updates relative to a regression task which has a unique estimated solution.

A consistent performance across our train, validation, and test set was observed. The MSE on the training set was 0.6330, while the validation and test sets recorded MSEs of 0.6212 and 0.6318. This means there was a solid generalization of the model without overfitting.

$$y = \theta_0 + \theta_1 x + \dots + \varepsilon \quad (6)$$

Here each θ_i term represents a coefficient that scales each standardized feature, giving some intuition to their effect on the model given positive or negative values.

D. Model 4: Random Forests Regression

The Random Forests Regression model is meant to capture non-linear relationships in the data. It constructs an ensemble of decision trees, and averages their results to make predictions, as seen in the equation below:

$$y = \frac{1}{N} \sum_{i=1}^N f_i(x) \quad (7)$$

We used a max depth of 10 with the same standardized features described in Section II, D. We chose to use the random forest regression because model because it is known to reduce overfitting.

Our model performed at an MSE of 0.5869 on the training set. The validation MSE was 0.6073 and the test MSE was 0.5890, all of which perform better than our linear regression model. This means the model was able to capture patterns in the data that the regression model did not incorporate with little overfitting.

E. Model 5: XGBoost

The XGBoost model uses an ensemble of weak decision trees in order to make predictions, where each decision tree helps to correct the errors of its predecessors. The novel aspect of XGBoost is that it uses a gradient boosting framework to minimize its objective function. To make a prediction, the XGBoost model sums up the predictions of each tree in the ensemble, as seen in the equation below:

$$y = \sum_{k=1}^n f_k(x_i) \quad (8)$$

XGBoost is known for its effectiveness in handling various types of data and was used to further refine our predictions. The model reached an MSE of 0.5810 on the training set and validation/test MSEs of 0.6092 and 0.5839. This demonstrates that XGBoost was able to slightly outperform Random Forest Regression in the training set.

F. Unsuccessful Attempts

The final features we used were the normalized average user rating, average restaurant rating, and the length of the review to ensure a low model complexity. For each of the models above, we also experimented with additional feature vectors. In particular, we also tried to incorporate other features from the review data including the time of the review, the location (latitude, longitude) of the review, and NLP features including sentiment of the review itself ranked on positivity, neutrality, and negativity. In addition, we tried to incorporate similarity metrics such as Jaccard. Nevertheless, we did not find an increase in performance from using these features along with our final features. We propose that since the dataset is uneven in terms of the number of reviews given by each user and for each restaurant, a mean based metric that isn't dependent on frequency of reviews was the most accurate predictor.

For a comparison of the strengths and weaknesses of the models, see *Section V*.

IV. LITERATURE REVIEW

A. Past Usage of Google Local Dataset

The Google Local Dataset we used was originally compiled in the papers [1] and [2], using data provided from Google maps. The first paper [1] used the actual text of reviews within the Google Local dataset for phrase mining. The second paper [2] generated a smaller annotated dataset from the Google Local dataset, which was used to generate "personalized showcases" of images that a user might like to see. Neither of these past applications of the dataset are related to our predictive task, but the dataset is useful for us nonetheless.

B. Similar Datasets

Other datasets exist for restaurant review prediction. One notable dataset is the Yelp dataset [3], which includes a variety of business attributes for millions of reviews. This dataset is limited to only 11 metropolitan areas, unlike the Google Local dataset which included data for entire states, allowing us to include restaurants outside of major metropolitan areas as well as those within them.

Another dataset is available called "Google Restaurants", also compiled from Google review data [4]. This dataset was produced in the "personalized showcase" paper above [2]. While this dataset initially seemed perfect for our use case since it had already filtered out restaurants from the rest of the businesses, this dataset didn't contain business meta-data, which was key the feature-engineering aspect of our approach.

C. Other Approaches to Restaurant Rating Prediction

Many attempts to predict restaurant ratings are based on natural language processing (NLP) techniques, using only the text of a review to predict that review's rating score. One such paper [5] approached this problem using the Yelp dataset referenced above, using an array of different feature representations (unigram, bigram, trigram, Latent Semantic Indexing) and multiple machine learning models (logistic regression, Naive Bayes classification, perceptrons, linear Support Vector Classification). This problem's predictive task is formulated differently to ours, as it is essentially a classification problem: given the text of a new review, predict that new review's rating score (classes 1 to 5). Importantly, this predictive task is not specific to any user, and does not predict with user information in mind.

The NLP-based approach in paper [6] used a sentence-level analysis of topic and sentiment information as part of their rating-prediction model. They classified each sentence in a review by these topical category and estimated sentiment, which was used to generate a 'text rating' for each review. This text rating could then be mapped to a predicted review score. Final results showed that integrating a text-derived rating for collaborative filtering (to achieve more personalized recommendations) lead to noticeable improvements in prediction accuracy. When compared against baselines of (i) predicting the average review score for that restaurant, and (ii) predicting the average review score for all restaurants of the same cuisine type (using business meta-data), this model performed significantly better. There was no comparison against baseline models that used other features.

Another approach to restaurant rating prediction which didn't use NLP is outlined in paper [7]. This model used multi-view clustering to extend simpler collaborative filtering models. Their approach was to generate multi-view clusters for both users and restaurants, and to predict new ratings (given a user and restaurant) by computing and combining the averages of k-Nearest Neighbors between the respective user and restaurant clusters.

Recent advances in the field of deep learning have been applied to review score prediction as well, with great success.

Using a metadata-based approach, a 2020 paper [8] implemented a 'Joint Deep Recommendation Model' (JRDM) for rating prediction. The JRDM leverages two parallel neural networks to learn low-level item and user latent representations separately, but higher-order feature interactions together using a shared last layer on the top, which is dense and fully-connected. Results showed that the JRDM performed better than other traditional recommendation algorithms like matrix factorization and hidden-factors-as-topics, and also was somewhat robust against cold-start and sparsity issues.

Another paper [9] identified a lack of research done which integrated review meta-data with more traditional content-based features and collaborative filtering methods. This paper developed an artificial neural network (ANN) for their hybrid approach. When trained on a single business category type (e.g., restaurants), this ANN was able to generalize well to other business categories and also to other users. The ANN was able to outperform a baseline user-based collaborative filtering method. The deep-learning approach requires much less memory than other recommendation algorithms like collaborative filtering which are memory-based, as it only needs to store the model's parameters. Notably, this work did not utilize geospatial data or review text at all in its methodology.

V. RESULTS AND CONCLUSIONS

We explored the Google Local Review (2021) Dataset of the Washington D.C. region filtering for restaurants. We chose to create a recommendation system through predicting the rating of a restaurant given a user review. Our investigation included the following models: a baseline predicting the mean, the bias model, the latent factor model, linear regression, random forests, and XGBoost. For the baseline model, we used the rating per restaurant as the feature to calculate the average. For the bias and LFM models, we used the user id, restaurant id, and the rating as inputs for the interaction data. For the linear regression model, random forests, and XGBoost, we created standardized feature vectors including the average rating each user gave to each restaurant, the average rating each restaurant received from each reviewer, and the the review length.

Model	MSE
Baseline	0.9641
LFM	0.7822
Bias	0.7688
Linear Regression	0.6318
Random Forests	0.5890
XGBoost	0.5839

Mean Squared Error (MSE) for Investigated Models

Our proposed model from our results in *Section III* show the XGBoost worked the best. This is due to its ability to not overfit as a feature of decision trees while still having the ability to fit non-linear boundaries. The random forests, with a similar model architecture, performed very similar to XGBoost and retains similar properties to the XGBoost model. Linear regression used features that were explicitly interaction

based and thus performed well, but lacked to encapsulate non-linear changes in the data. Nevertheless, the two models did not have a large difference between them. Next, both the bias model and the LFM performed very similarly with the LFM performing worse. We infer that the bias performed worse due to a lack of interaction data in the model as it predicts based on user and restaurant rating deviations from the global bias term. Our inference for the LFM performing worse than the bias is due to the sparsity of the user-restaurant interactions across the dataset. If we think about the matrix describing their interactions and try to decompose it, there will be a lot of estimations on majority zeroed out rows and columns leading to little information being stored in the latent dimensions, and thus creating noise in the prediction. Finally, the baseline model serves purely as a metric to beat and essentially represent the variance within the dataset. We used the default XGBoost depth of 6 and used a learning rate of 0.1 which gave us the best results. From our interpretation of the models parameters we found that that a less complex model due to the smaller depth but with a moderately aggressive learning rate allowed for non-linear boundaries to be found without overfitting.

We didn't consider any of our models to fail as they all outperformed the trivial baseline significantly. Nevertheless, it is important to mention that feature engineering was the main factor in our models performance. As opposed to constructing feature vectors from raw user and restaurant data, our intuition for interactions between the user and the restaurant significantly improved our predictions with little overfitting. For more information about feature engineering visit *Section II, D* and *Section III, F*.

For future work, the literature review papers serve as a stepping stone for this project which include a deep learning approach such as the JRDM model for deep latent representations [8] and use feature engineering with more predictive sentiment using current NLP practices [6]. Nevertheless, our model performed very well using our simple and low-dimensional feature representations, reinforcing our assumptions for feature engineering and model creation from the course.

REFERENCES

- [1] J. Li, J. Shang, and J. McAuley, "UCTopic: Unsupervised Contrastive Learning for Phrase Representations and Topic Mining," Annual Meeting of the Association for Computational Linguistics (ACL), May 2022.
- [2] A. Yan, Z. He, J. Li, T. Zhang, J. McAuley, "Personalized Showcases: Generating Multi-Modal Explanations for Recommendations," The 46th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR), 2023.
- [3] Yelp Dataset, "Yelp Dataset: Business, Reviews, and User Data," <https://www.yelp.com/dataset>
- [4] J. McAuley, "Datasets for Research on Recommender Systems," UC San Diego, <https://cseweb.ucsd.edu/jmcauley/datasets.html#google-restaurants>
- [5] N. Asghar, "Yelp dataset challenge: Review rating prediction," arXiv preprint arXiv:1605.05362, 2016.
- [6] G. Ganu, N. Elhadad, A. Marian, "Beyond the stars: improving rating predictions using review text content," In WebDB Vol. 9, pp. 1-6, June 2009

- [7] L. Pradhan, C. Zhang and P. Chitrakar, "Multi-view Clustering in Collaborative Filtering Based Rating Prediction," 2016 IEEE Tenth International Conference on Semantic Computing (ICSC), Laguna Hills, CA, USA, 2016, pp. 250-253, doi: 10.1109/ICSC.2016.40.
- [8] Z.Y. Khan, Z. Niu, and A. Yousif. "Joint deep recommendation model exploiting reviews and metadata information," *Neurocomputing* 402, 2020.
- [9] T.K.Paradarami, N.D. Bastian, and J.L. Wightman. "A hybrid recommender system using artificial neural networks," *Expert Systems with Applications* 83, 2017.